

IMD0029 - ESTRUTURA DE DADOS BÁSICAS I - T06

Trabalho Prático 1 Und. 2 – Tipos Abstratos de Dados (TADs)

Elton Gustavo da Silva Freitas - 20250024590

Introdução

Os Tipos Abstratos de Dados (TADs) são estruturas fundamentais, pois permitem organizar, armazenar e manipular informações de maneira eficiente e organizada. Um TAD define o comportamento de uma estrutura de dados por meio de operações específicas, sem necessariamente expor os detalhes de sua implementação interna. Dessa forma, é possível separar a lógica de utilização da estrutura da forma como ela é construída.

Neste trabalho, foram estudados e implementados três diferentes Tipos Abstratos de Dados: o TAD Sequência, o TAD Conjunto e o TAD ArraySequence, utilizando a linguagem C++. O desenvolvimento das estruturas foi realizado com o uso de ponteiro e, alocação dinâmica de memória, sem a utilização de bibliotecas prontas que substituam as implementações solicitadas.

Além da implementação prática, este trabalho também busca analisar o funcionamento e a complexidade das principais operações realizadas em cada estrutura, demonstrando a importância dos TADs na construção de sistemas eficientes e organizados.

TAD SEQUÊNCIA

O TAD Sequência é uma estrutura de dados utilizada para armazenar elementos de forma ordenada, permitindo que cada item seja acessado por meio de uma posição específica. Nesse tipo de estrutura, os elementos mantêm uma relação de sequência, ou seja, existe uma ordem definida entre eles.

As principais operações desse TAD incluem inserir elementos em posições específicas, remover elementos da sequência e acessar valores armazenados. Essas funcionalidades tornam o TAD Sequência uma estrutura importante para diversas aplicações que necessitam de organização e manipulação ordenada de dados.

Abaixo estão duas capturas de tela com os arquivos de cabeçalho **no.h** e **sequencia.h**, onde estão definidos todos os atributos e métodos utilizados para criar esse TAD.

```
#pragma once

class No{

private:
    int valor;
    No* proximo;
public:
    No(int v, No* no);
    int getValor();
    No* getProximo();
    void setProximo(No* no);
};
```

```
#pragma once

#include <iostream>
#include "no.h"

class Sequencia{
private:
    int quantidade = 0;
    No* inicio;
public:
    Sequencia();
    bool insert(int pos, int elem);
    bool remove(int pos);
    void get(int pos);
    void print();
    int getQuantidade();
};
```

Como pode ser observado, em vez de utilizar vetores dinâmicos, resolvi seguir o outro modelo presente no material, que é uma sequência baseada em uma lista encadeada de nós, usando esse método, na declaração da classe sequência, precisamos apenas saber qual a quantidade de Nós que estão na sequência e qual o primeiro Nó da sequência, e a partir dele é possível chegar até o último, que será caracterizado por apontar para o vazio, o que indica o fim de sequência. Abaixo está a implementação no arquivo **no.cpp**:

```
#include "no.h"

No::No(int v, No* no){
    valor = v;
    proximo = no;
}

int No::getValor(){
    return valor;
}

No* No::getProximo(){
    return proximo;
}

void No::setProximo(No* no){
    proximo = no;
}
```

Construtor: recebe como parâmetro um valor e o endereço de memória de um **No**, e define esses valores nos atributos.

getValor e getProximo: Apenas retornam os valores de seus respectivos atributos.

setProximo: Recebe um endereço de memória de um **No** como parâmetro e define o atributo privado **proximo** como esse **No**, esse método de definir o próximo da sequência é que permite criar essa TAD de forma tão organizada.

Abaixo mostrarei alguns trechos de código do arquivo **sequencia.cpp**, ou seja a implementação do TAD.

```
Sequencia::Sequencia(){
    quantidade = 0;
    inicio = nullptr;
}
```

O construtor apenas define a quantidade de elementos igual a 0 e define que o início é nulo, pois ele ainda não foi criado.

Abaixo temos o início do principal método, o **insert**, ele recebe uma posição e um elemento para ser inserido nessa posição, na imagem abaixo temos o caso específico de quando a quantidade de elementos da sequência é 0, forçamos o usuário na sua primeira vez ao inserir um elemento, o colocar na posição 1, assim que validado é criado um **nó** com o elemento passado por parâmetro e apontando para o vazio, e finalmente definimos o início da sequência como esse **nó**, e por fim controlamos a quantidade de elementos colocando **quantidade++**.

```
bool Sequencia::insert(int pos, int elem){
    if(quantidade == 0){
        if(pos != 1){
            std::cout << std::endl << "A lista está vazia, adicione um elemento na posição 1" << std::endl;
            return false;
        }
        No* no = new No(elem, nullptr);
        inicio = no;
        std::cout << std::endl << no->getValor() << " adicionado com sucesso na posição " << 1 << std::endl;
        quantidade++;
        return true;
    } else{
```

Dentro do **else** acima, temos mais estruturas de condição, agora estamos dentro do cenário que quantidade não é mais nula, então temos que validar algumas operações a fim que o código não quebre.

A primeira é verificar se a posição solicitada existe dentro da sequência, se ela for menor ou igual a 0 ou maior que o próximo elemento da sequência, ela não existe, portanto retorna **false** diretamente, encerrando o método.

E agora temos dois casos específicos tratados quando a posição é válida, quando queremos adicionar no lugar do primeiro elemento e quando queremos adicionar como próximo elemento.

```

} else{
    if(pos <= 0 || pos > quantidade+1){
        std::cout << std::endl << "Posição inválida!" << std::endl;
        return false;
    }else{
        if(pos == 1){
            No* no = new No(elem, inicio);
            inicio = no;
            std::cout << std::endl << no->getValor() << " adicionado com sucesso na posição " << 1 << std::endl;
            quantidade++;
        }else if(pos == quantidade + 1){
            No* no = new No(elem, nullptr);
            No* atual = inicio;

            while (atual->getProximo() != nullptr) {
                atual = atual->getProximo();
            }

            atual->setProximo(no);
            quantidade++;
            std::cout << std::endl << no->getValor() << " adicionado com sucesso na posição " << quantidade << std::endl;
        } else {

```

Para adicionar um novo primeiro elemento, é bem simples, basta criar um novo **nó** com seu valor apontando para o **inicio** atual, e definir o **nó** recém criado como novo **inicio** na sequência, e por fim **quantidade++** para controlar a quantidade de elementos.

Para adicionar um elemento ao final da lista, cria-se um **nó** apontando para o vazio, e também um **nó** auxiliar para ser usado em um **while** iterando até chegar no último **nó** definido da sequência, usando a condição de parada **atual->getProximo != nullptr**, assim ele verifica se o próximo é o último, e quando finalmente chegar nele, é usada a função **atual->setProximo(no)** passando como parâmetro o **nó** criado e iterando a quantidade.

Agora que os dois casos específicos foram explicados, vamos para o último **else** no caso geral, onde adicionamos elementos em posições no meio da sequência:

```

} else {
    int aux = 1;
    No* no = new No(elem, nullptr);
    No* atual = inicio;
    while (aux < pos-1){
        atual = atual->getProximo();
        aux++;
    }

    no->setProximo(atual->getProximo());
    atual->setProximo(no);

    std::cout << std::endl << no->getValor() << " adicionado na posição " << aux+1 << std::endl;

    quantidade++;
}

```

O método é bem parecido, mas precisamos de uma variável auxiliar, criamos um **nó** apontando pro **nullptr (vazio)** e um **nó** auxiliar começando do **inicio**, usamos o **while** para percorrer até a posição exatamente anterior onde queremos adicionar o elemento, e realizamos duas ações:

- Fazemos o novo **nó** apontar para o endereço de memória que o **nó** que está na posição que queremos aponta, e agora os dois estão apontando para o mesmo lugar.
- Agora podemos fazer o **nó** que está na posição desejada apontar para o novo **nó** criado, assim inserimos o **nó** que acabamos de criar no meio de dois **nós**, assim resolvendo o caso geral.

```
void Sequencia::print(){
    std::cout << std::endl << "(PRINT)" << std::endl;
    std::cout << "Sequência: ";

    if (inicio == nullptr || quantidade == 0) {
        std::cout << "Lista vazia!";
    }

    No* atual = inicio;

    while (atual != nullptr) {
        std::cout << atual->getValor() << " ";
        atual = atual->getProximo();
    }

    std::cout << std::endl;
    std::cout << "quantidade: " << quantidade << std::endl << std::endl;
}
```

No método **print**, apenas usamos a mesma estratégia de percorrer a sequência e mostramos no terminal usando o **std::cout** formatado de forma a ficar mais agradável a leitura.

Por fim, abaixo temos o método **remove**, são usadas as mesmas validações do **insert** para o fluxo do método, se a posição é válida, remover primeiro elemento, remover último elemento e remover posição geral no meio da sequência, para remover apenas redirecionamos os **nós** envolvidos para o novo **proximo** deles e removemos com **delete[]** o **nó** que será removido, decrementando na quantidade de elementos.

```

bool Sequencia::remove(int pos){
    if(pos <= 0 || pos > quantidade){
        std::cout << std::endl << "Posição inválida!" << std::endl;
        return false;
    }else{
        No* atual = inicio;
        if(pos == 1){
            inicio = atual->getProximo();
            std::cout << std::endl << atual->getValor() << " removido com sucesso da posição " << pos << std::endl;
            delete[] atual;
        } else if(pos == quantidade){
            int aux = 1;
            while(aux < pos-1){
                atual = atual->getProximo();
                aux++;
            }
            std::cout << std::endl << atual->getProximo()->getValor() << " removido com sucesso da posição " << pos << std::endl;
            delete[] atual->getProximo();
            atual->setProximo(nullptr);
        } else{
            int aux = 1;
            while(aux < pos-1) {
                atual = atual->getProximo();
                aux++;
            }
            No* no_aux = atual->getProximo();
            atual->setProximo(no_aux->getProximo());
            std::cout << std::endl << no_aux->getValor() << " removido com sucesso da posição " << pos << std::endl;
            delete[] no_aux;
        }
        quantidade--;
        return true;
    }
}

```

Para compilação criei um Makefile para facilitar na execução dos testes, após usar o comando make no terminal tudo é compilado corretamente:

```

elton@pop-os:~/dev/edb/trabalho_u2/tad_sequencia$ make
g++ -Wall -std=c++17 -c main.cpp -o main.o
g++ -Wall -std=c++17 -c no.cpp -o no.o
g++ -Wall -std=c++17 -c sequencia.cpp -o sequencia.o
g++ -Wall -std=c++17 main.o no.o sequencia.o -o main
elton@pop-os:~/dev/edb/trabalho_u2/tad_sequencia$

```

Ao usar o comando **make run** após a compilação, irá se deparar com um menu criado no arquivo **main.cpp** que possui algumas opções para interagir com a sequência, mas para fins desse trabalho apenas a opção 4 nos importa.

```

elton@pop-os:~/dev/edb/trabalho_u2/tad_sequencia$ make run
./main

Digite o que fazer na sequencia:
1. insert(pos, elem) | 2. remover(pos) | 3. get(pos) | 4. teste mínimo sugerido | 5. encerrar sistema

(PRINT)
Sequência: Lista vazia!
quantidade: 0

Digite a opção:

```


Abaixo temos o código teste e sua execução no terminal.

```
case 4:
    for (int i = 1; i <= 10; i++){
        s.insert(i, i*2);
    }

    s.print();

    s.remove(5);
    s.remove(6);

    s.print();
    break;
```

Teste mínimo sugerido

- Inserir 10 elementos;
- Remover 2 elementos do meio;
- Exibir a sequência antes e depois da remoção.

```
Digite a opção: 4
2 adicionado com sucesso na posição 1
4 adicionado com sucesso na posição 2
6 adicionado com sucesso na posição 3
8 adicionado com sucesso na posição 4
10 adicionado com sucesso na posição 5
12 adicionado com sucesso na posição 6
14 adicionado com sucesso na posição 7
16 adicionado com sucesso na posição 8
18 adicionado com sucesso na posição 9
20 adicionado com sucesso na posição 10

(PRINT)
Sequência: 2 4 6 8 10 12 14 16 18 20
quantidade: 10

10 removido com sucesso da posição 5
14 removido com sucesso da posição 6

(PRINT)
Sequência: 2 4 6 8 12 16 18 20
quantidade: 8
```

TAD CONJUNTO

O TAD Conjunto é uma estrutura de dados abstrata inspirada no conceito matemático de conjuntos. Ao contrário do TAD Sequência, sua principal característica é a ausência de duplicidade e a inexistência de uma ordem linear obrigatória entre os elementos. Isso significa que um item só pode existir uma única vez dentro da estrutura e que a sua posição física na memória não dita uma hierarquia posicional para o usuário.

As operações fundamentais desse TAD envolvem a manipulação de elementos únicos, como inserir sem duplicar, remover e verificar a pertinência (se um elemento pertence ou não ao conjunto, além das clássicas operações da teoria dos conjuntos, como a União e a Interseção).

Essas propriedades tornam o TAD Conjunto uma ferramenta essencial no desenvolvimento de algoritmos que exigem filtragem de dados repetidos, testes rápidos de presença e agrupamentos lógicos, sendo amplamente aplicado em sistemas de banco de dados, motores de busca e análise de redes.

```
#pragma once

class Conjunto{

private:
    int capacidade;
    int* elementos;
    int quant_elementos;

public:
    Conjunto();
    void add(int elem);
    void remove(int elem);
    bool contains(int elem);
    Conjunto sets_union(Conjunto a, Conjunto b);
    Conjunto sets_intersection(Conjunto a, Conjunto b);
    void print();
    void redimensionar();
};
```

Nesse arquivo de cabeçalho temos 3 atributos privados importantes, a capacidade do conjunto, um ponteiro de um **array** de elementos e a quantidade de elementos nesse conjunto.

A lógica trabalhada aqui será muito simples, um **array** dinâmico que usa dos métodos ao lado para simular um conjunto matemático.

```
Conjunto::Conjunto(){
    capacidade = 3;
    elementos = new int[capacidade];
    quant_elementos = 0;
}

bool Conjunto::contains(int elem){
    for (int i = 0; i < quant_elementos; i++){
        if(elem == elementos[i]){
            return true;
        }
    }
    return false;
}
```

No construtor, definidos a capacidade inicial como 3, criamos o vetor dinâmico e definimos a quantidade de elementos como 0.

No método **contains**, já que por dentro é um **array**, apenas o percorremos com um **for** usando a quantidade de elementos e retornamos **true** ou **false**.

```
void Conjunto::redimensionar(){
    capacidade*=2;
    int* novos_elementos = new int[capacidade];

    for (int i = 0; i < quant_elementos; i++){
        novos_elementos[i] = elementos[i];
    }

    delete[] elementos;
    elementos = novos_elementos;
}

void Conjunto::print(){
    std::cout << "{";
    for (int i = 0; i < quant_elementos; i++){
        if(i < quant_elementos - 1){
            std::cout << elementos[i] << ", ";
        }else{
            std::cout << elementos[i];
        }
    }
    std::cout << "}" << std::endl;
}
```

Para deixar o **array** dinâmico, criei um método chamado **redimensionar**, ele cria um novo ponteiro de um **array** temporário com o dobro da capacidade, itera um **for** para redefinir o local do elementos na memória, apaga os dados antigos e realoca o nome para o original.

A função **print** apenas usada do **std::cout** de forma inteligente com um **for** para exibir o conjunto formatado para o usuário.


```

void Conjunto::add(int elem){
    if(quant_elementos == capacidade){
        redimensionar();
    }

    if(contains(elem) == false){
        elementos[quant_elementos] = elem;
        quant_elementos++;
    }
}

void Conjunto::remove(int elem){
    if(contains(elem) == true){
        for (int i = 0; i < quant_elementos; i++){
            if(elem == elementos[i]){
                int aux = elementos[quant_elementos-1];
                elementos[quant_elementos-1] = elem;
                elementos[i] = aux;
            }
        }
        quant_elementos--;
    }
}

```

Para adicionar elementos, sempre verificamos se é necessário redimensionar ou não.

Usamos um **if** com o método já explicado **contains** para verificar se o elemento já existe no conjunto, se não existir, adicionamos como último elemento e iteramos a quantidade.

Já para remover, verificamos se existe, se sim, então percorremos a lista até achá-lo e por meio de uma variável auxiliar trocamos ele de lugar com o último da lista e decrementamos a quantidade de elementos do conjunto.

Agora que a classe já emula perfeitamente o comportamento de um conjunto, agora falta apenas implementar operações entre conjuntos, nesse trabalho foi solicitado apenas a interseção e união, com os nomes **intersection** e **union** respectivamente, mas infelizmente **union** é palavra reservada no C++ então tomei liberdade para renomear os métodos, o código está abaixo:

```

Conjunto Conjunto::sets_union(Conjunto a, Conjunto b){
    Conjunto uni;
    for (int i = 0; i < a.quant_elementos; i++){
        uni.add(a.elementos[i]);
    }

    for (int i = 0; i < b.quant_elementos; i++){
        uni.add(b.elementos[i]);
    }

    return uni;
}

Conjunto Conjunto::sets_intersection(Conjunto a, Conjunto b){
    Conjunto inter;

    for (int i = 0; i < a.quant_elementos; i++){
        for (int j = 0; j < b.quant_elementos; j++){
            if(a.contains(b.elementos[j])){
                inter.add(b.elementos[j]);
            }
        }
    }

    return inter;
}

```

Para união foi simples, como o método retorna um objeto da classe conjunto, então criei ele e fiz dois **for** iterando nos conjuntos **a** e **b** que foram passados como parâmetro, já que pela natureza dos conjuntos não pode haver elementos repetidos, só isso já foi o bastante para representar a união.

Para interseção foi um pouco mais complicado, foi necessário fazer um **for** dentro do outro para verificar se cada elemento do conjunto **a** contém algum elemento do conjunto **b**, se sim, então seria adicionado no objeto criado e no fim retornando ele.

```

int main(){
    Conjunto A;
    Conjunto B;
    A.add(1);
    A.add(2);
    A.add(3);
    A.add(5);
    B.add(3);
    B.add(4);
    B.add(5);
    B.add(6);
    std::cout << "A = ";
    A.print();
    std::cout << "B = ";
    B.print();
    Conjunto uni = uni.sets_union(A, B);
    std::cout << "A ∪ B = ";
    uni.print();
    Conjunto inter = uni.sets_intersection(A, B);
    std::cout << "A ∩ B = ";
    inter.print();

    return 0;
}

```

Teste mínimo sugerido

A = {1, 2, 3, 5}

B = {3, 4, 5, 6}

Exibir:

- União
- Interseção

```

A ∩ B = {3, 5}
elton@pop-os:~/dev/edb/trabalho_u2/tad_conjunto$ make run
g++ -Wall -std=c++17 -c main.cpp -o main.o
g++ -Wall -std=c++17 main.o conjunto.o -o main
./main
A = {1, 2, 3, 5}
B = {3, 4, 5, 6}
A ∪ B = {1, 2, 3, 5, 4, 6}
A ∩ B = {3, 5}

```

TAD ARRAY SEQUENCE

O TAD Array Sequence é uma estrutura de dados que funciona como uma lista organizada de elementos, onde a ordem deles realmente importa. Ela usa um vetor comum (um array) por baixo dos panos para guardar as informações. A grande vantagem dessa estrutura é que você pode acessar qualquer elemento diretamente sabendo a posição dele (o índice), de um jeito muito rápido e direto.

As operações mais comuns envolvem colocar novos elementos em qualquer lugar da lista, remover itens que já estão lá e alterar ou ler o valor de uma posição específica. Como o tamanho do vetor fixo pode acabar, essa estrutura também é inteligente o suficiente para aumentar de tamanho sozinha quando fica cheia, criando mais espaço para novos dados.

Por ser simples e muito eficiente para ler informações, esse modelo é a base de recursos que usamos todos os dias na programação, como as listas dinâmicas. Ele é perfeito para situações em que precisamos manter uma fila de tarefas organizada, históricos de navegação ou qualquer lista onde a ordem de chegada dos dados precisa ser respeitada.

Abaixo temos o arquivo **arraysequence.h**

```
#pragma once

class ArraySequence{

private:
    int capacidade;
    int* array;
    int tamanho;

public:
    ArraySequence();
    void pushBack(int value);
    void pushFront(int value);
    void insert(int pos, int value);
    void remove(int pos);
    bool find(int value);
    int size();
    void print();
    void redimensionar();
};
```

A classe tem 3 atributos que serão usados durante o gerenciamento do comportamento de um ArraySequence, a capacidade do **Array**, o tamanho atual dele e um ponteiro de inteiros para gerenciar o **array** dinâmico.

Implementar o ArraySequence me permitiu compreender na prática o funcionamento das operações de inserção e remoção em posições específicas do vetor. Para adicionar elementos no início ou no meio da estrutura, foi necessário deslocar os valores existentes para abrir espaço para o novo elemento.

Da mesma forma, ao remover um valor, os elementos posteriores precisam ser movidos para evitar espaços vazios no array. Esse comportamento mostrou como estruturas sequenciais baseadas em vetor realizam o gerenciamento interno dos dados armazenados.

Além dos métodos requisitados, adicionei o método **redimensionar** também pois assim como a **TAD CONJUNTO**, pois foi a melhor forma que imaginei para gerenciar **arrays** dinâmicos.

```

ArraySequence::ArraySequence(){
    capacidade = 3;
    array = new int[capacidade];
    tamanho = 0;
}

void ArraySequence::redimensionar(){
    capacidade *= 2;
    int* novo_array = new int[capacidade];

    for (int i = 0; i < tamanho; i++){
        novo_array[i] = array[i];
    }

    delete[] array;
    array = novo_array;
}

int ArraySequence::size(){
    return tamanho;
}

void ArraySequence::print(){
    std::cout << std::endl << "ArraySequence = [";
    for(int i = 0; i < tamanho; i++){
        if(i < tamanho - 1){
            std::cout << array[i] << ", ";
        } else{
            std::cout << array[i];
        }
    }
    std::cout << "]" << std::endl;
}

```

Para o construtor, definimos a capacidade inicial para 3, criamos o vetor dinâmico e definimos seu tamanho para 0.

Para redimensionar utiliza-se a mesma técnica usada no TAD CONJUNTO.

O método **size** retorna o atributo tamanho do vetor.

O método **print** apenas formata a saída percorrendo o **array** com um **for** para ficar mais agradável para o usuário.

```

bool ArraySequence::find(int value){
    for (int i = 0; i < tamanho; i++){
        if(array[i] == value){
            std::cout << "Elemento [" << value << "] encontrado na posição " << i+1 << std::endl;
            return true;
        }
    }
    std::cout << "Elemento [" << value << "] não encontrado na sequência" << std::endl;
    return false;
}

```

Acima temos o método **find**, o qual apenas percorre o vetor e compara cada elemento com o **value** passado como parâmetro, se forem iguais, retorna **true**, se percorrer todo o vetor e não achar o elemento, retorna **false**.


```

void ArraySequence::pushBack(int value){
    if(tamanho == capacidade){
        redimensionar();
    }

    array[tamanho] = value;
    tamanho++;
}

void ArraySequence::pushFront(int value){
    if(tamanho == capacidade){
        redimensionar();
    }

    for (int i = tamanho - 1; i >= 0; i--){
        array[i+1] = array[i];
    }

    array[0] = value;
    tamanho++;
}

```

Os dois métodos ao lado adicionam respectivamente elementos ao final e ao começo do **array**, mas antes de qualquer ação eles verificam se é necessário ou não redimensionar a capacidade.

Tendo verificado isso, as formas de adicionar são simples, no **pushBack**, usamos o atributo **tamanho** como índice e adicionamos o **value**, depois incrementamos **tamanho**, já para o **pushFront**, usando um **for**, movemos todos os elementos para direita começando do final, e depois definidos o índice 0 como o **value**, e depois incrementa-se o **tamanho**.

```

void ArraySequence::remove(int pos){
    if(pos <= 0 || pos > tamanho){
        std::cout << "Posição Inválida!" << std::endl;
        return;
    }

    if(pos == tamanho){
        tamanho--;
    } else {
        for (; pos < tamanho; pos++){
            array[pos - 1] = array[pos];
        }
        tamanho--;
    }
}

```

O método **remove** após a verificação se a posição é válida ou não, caso a posição passada por parâmetro seja igual ao tamanho do **array**, basta decrementar o **tamanho** em 1. Para o caso geral, basta usar um **for** começando da posição até o tamanho para fazer o antecessor receber o valor do atual.

Por fim resta apenas a função **insert**, a qual antes de tudo faz duas verificações, primeiro verifica se a posição que o elemento será adicionado é válida, caso seja, já que está adicionando elementos, o método verifica se o tamanho é igual a capacidade, se sim, redimensiona-se o **array** para suportar a quantidade de elementos do **array**.


```

void ArraySequence::insert(int pos, int value){
    if(pos <= 0 || pos > tamanho + 1){
        std::cout << std::endl << pos << " é uma posição inválida!" << std::endl;
        return;
    }

    if(tamanho == capacidade){
        redimensionar();
    }

    if(pos == 1){
        pushFront(value);
    } else if (pos == tamanho + 1) {
        pushBack(value);
    } else {
        for (int i = tamanho; i > pos-1; i--){
            array[i] = array[i - 1];
        }
        array[pos - 1] = value;
        tamanho++;
    }
}
}

```

Já que essa função considera a posição para adicionar elementos, nos casos específicos de primeiro e último elemento, podemos apenas usar os métodos **pushFront** e **pushBack** já criados, e sobra apenas o caso geral, onde se cria um inteiro **i** recebendo o tamanho, e enquanto for maior que [posição -1], decremente ele, assim podemos simplesmente fazer o atual receber o antecessor, e depois basta definir na posição desejada diretamente o **value**, não esquecendo de incrementar o tamanho após a operação.

```

int main(){

    ArraySequence array;

    std::cout << "Inserir os valores: [10, 20, 30, 40]";
    array.pushBack(10);
    array.pushBack(20);
    array.pushBack(30);
    array.pushBack(40);

    array.print();

    std::cout << std::endl << "Inserir um elemento no início";
    array.pushFront(0);

    array.print();

    std::cout << std::endl << "Inserir um elemento no meio";
    array.insert(4, 25);

    array.print();

    std::cout << std::endl << "Remover um elemento";
    array.remove(6);

    array.print();

    std::cout << std::endl << "Exibir a estrutura final";
    array.print();

    return 0;
}

```

Ao lado está o **main** com os testes requeridos no documento, e abaixo a execução desse código no terminal.

```

elton@pop-os:~/dev/edb/trabalho_u2/tad_array_sequence$ make run
g++ -Wall -std=c++17 -c main.cpp -o main.o
g++ -Wall -std=c++17 -c arraysequence.cpp -o arraysequence.o
g++ -Wall -std=c++17 main.o arraysequence.o -o main
./main
Inserir os valores: [10, 20, 30, 40]
ArraySequence = [10, 20, 30, 40]

Inserir um elemento no início
ArraySequence = [0, 10, 20, 30, 40]

Inserir um elemento no meio
ArraySequence = [0, 10, 20, 25, 30, 40]

Remover um elemento
ArraySequence = [0, 10, 20, 25, 30]

Exibir a estrutura final
ArraySequence = [0, 10, 20, 25, 30]
elton@pop-os:~/dev/edb/trabalho_u2/tad_array_sequence$

```

Tabela de Complexidade

Estrutura / Operação	Melhor Caso	Caso Médio	Pior Caso
TAD SEQUENCIA			
insert(pos, elem) no início	$O(1)$	—	—
insert(pos, elem) no meio/fim	$O(n)$	$O(n)$	$O(n)$
remove(pos) no início	$O(1)$	—	—
remove(pos) no meio/fim	$O(n)$	$O(n)$	$O(n)$
get(pos)	$O(1)$	$O(n)$	$O(n)$
print()	$O(n)$	$O(n)$	$O(n)$
TAD Conjunto			
contains(elem)	$O(1)$	$O(n)$	$O(n)$
add(elem)	$O(1)$	$O(n)$	$O(n)$
remove(elem)	$O(1)$	$O(n)$	$O(n)$
sets_union(A, B)	$O(n + m)$	$O(n \times m)$	$O(n \times m)$
sets_intersection(A, B)	$O(n \times m)$	$O(n \times m)$	$O(n \times m)$
TAD ArraySequence			
pushBack(value)	$O(1)$	$O(1)$	$O(n)$
pushFront(value)	$O(n)$	$O(n)$	$O(n)$
insert(pos, value)	$O(1)$	$O(n)$	$O(n)$
remove(pos)	$O(1)$	$O(n)$	$O(n)$
find(value)	$O(1)$	$O(n)$	$O(n)$
print()	$O(n)$	$O(n)$	$O(n)$

Conclusão

O desenvolvimento deste trabalho permitiu compreender na prática o funcionamento das principais Estruturas de Dados Abstratas (TADs), bem como suas diferenças em relação à forma de armazenamento dos dados. Durante a implementação das estruturas **Sequencia**, **Conjunto** e **ArraySequence**, foi possível observar como cada operação possui custos computacionais distintos dependendo da organização interna dos elementos.

Por fim, o trabalho contribuiu significativamente para o entendimento prático dos conceitos estudados em sala de aula, reforçando conhecimentos sobre ponteiros, alocação dinâmica de memória, manipulação de **arrays** e análise de eficiência algorítmica.

Link para repositório do trabalho:

<https://github.com/eltongustavo/trabalho-tad-edb>